Урок 7.3 — DEMO GUIDE для ведучого

> **Як користуватися цим файлом:**
>
> - Ідеш по слайдах (`lesson_7_3_slides.md`)
> - Коли бачиш слайд із міткою **«→ Демонстрація»** — відкриваєш цей файл і виконуєш відповідний блок
> - Після демонстрації повертаєшся до слайдів і продовжуєш
>
> **Середовище:** Docker, сервіс `php` (`docker compose exec php ...`)
> **Сервер:** `http://localhost` (порт 80, вже запущений у контейнері)
> **Тести:** SQLite `:memory:` (налаштовано в `phpunit.xml`, не потребує окремої БД)

ПІДГОТОВКА (зробити до митів)

```
# 1. Переконайся, що контейнери запущені
docker compose up -d

# 2. Застосувати нові міграції
docker compose exec php php artisan migrate

# 3. Перевірити що нові тести зелені
docker compose exec php php artisan test tests/Feature/Api/V1/ --no-coverage
```

Очікуваний результат тестів:

```
PASS  Tests\Feature\Api\V1\CreatePaymentTest         (2 tests)
PASS  Tests\Feature\Api\V1\CreatePaymentValidationTest (4 tests)
PASS  Tests\Feature\Api\V1\ShowPaymentNotFoundTest    (2 tests)

Tests: 8 passed (49 assertions)
```

БЛОК 2 → Слайд 6a

Демонстрація: openapi.yaml — формальний опис контракту

Що показати: файл `public/openapi.yaml`

Відкрий в IDE: `public/openapi.yaml`

Прокоментуй три секції:

1. Ендпоінт POST /api/v1/payments:

```yaml
paths:
  /api/v1/payments:
    post:
      summary: Create a new payment
      requestBody:
```

```
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/CreatePaymentRequest'
        responses:
          '201':
            description: Payment created successfully
          '422':
            description: Validation error
```

2. Схема вхідного запиту:

```
CreatePaymentRequest:
  required: [account_id, amount, currency]
  properties:
    account_id:  { type: integer }
    amount:      { type: string, pattern: '^\d+(\.\d{1,2})?$' }
    currency:    { type: string, enum: [UAH, USD, EUR] }
    description: { type: string, nullable: true }
```

3. Схема помилки 422:

```
ValidationError:
  properties:
    message: { type: string }
    errors:
      type: object
      additionalProperties:
        type: array
        items: { type: string }
```

Ключовий меседж: «Цей YAML — формальний контракт. Клієнт може за ним згенерувати TypeScript-типи або форми валідації без єдиного слова від бекенд-розробника.»

БЛОК 2b → Слайд 6b

Демонстрація: Swagger UI — як переглянути OpenAPI в браузері

> Є два підходи: **мануально** (без пакетів, через статичний HTML) і **через пакет** (`darkaonline/l5-swagger`).

Варіант 1 — Мануально (статичний HTML + CDN)

Найшвидший спосіб: один HTML-файл, нічого не встановлювати.

Крок 1. Створити файл `public/swagger-ui.html`:

```
<!DOCTYPE html>
<html lang="en">
<head>
```

```
    <meta charset="UTF-8" />
    <title>Swagger UI</title>
    <link rel="stylesheet" href="https://unpkg.com/swagger-ui-dist/swagger-ui.css" />
  </head>
  <body>
    <div id="swagger-ui"></div>
    <script src="https://unpkg.com/swagger-ui-dist/swagger-ui-bundle.js"></script>
    <script>
      SwaggerUIBundle({
        url: "/openapi.yaml",
        dom_id: "#swagger-ui",
        presets: [SwaggerUIBundle.presets.apis,
  SwaggerUIBundle.SwaggerUIStandalonePreset],
        layout: "BaseLayout",
      });
    </script>
  </body>
  </html>
```

Крок 2. Переконатися, що сервер запущений:

```
docker compose up -d
```

Крок 3. Відкрити у браузері:

```
http://localhost/swagger-ui.html
```

> `public/openapi.yaml` вже доступний як статичний файл через nginx — додаткових маршрутів не потрібно.

Ключовий меседж: «Swagger UI — це просто JavaScript-додаток. Він читає будь-який OpenAPI YAML/JSON за URL і рендерить інтерактивну документацію. Пакети не обов'язкові.»

Варіант 2 — Через пакет `darkaonline/l5-swagger`

Пакет дозволяє генерувати OpenAPI прямо з PHP-атрибутів у коді (анотацій), але також вміє підключити готовий YAML.

Крок 1. Встановити пакет:

```
docker compose exec php composer require darkaonline/l5-swagger
docker compose exec php php artisan vendor:publish --provider
"L5Swagger\L5SwaggerServiceProvider"
```

Це створить `config/l5-swagger.php` і `resources/views/vendor/l5-swagger/`.

Крок 2. У `config/l5-swagger.php` змінити три рядки, щоб пакет показував **готовий** `public/openapi.yaml` замість генерації з PHP-анотацій:

```
// documentations → default → paths
'docs_yaml'              => 'openapi.yaml',       // було: 'api-docs.yaml'
'format_to_use_for_docs' => 'yaml',               // було: 'json'

// defaults → paths
'docs' => public_path(),                          // було: storage_path('api-docs')
```

Крок 3. Скинути кеш конфігу:

```
docker compose exec php php artisan config:clear
```

Крок 4. Відкрити у браузері:

```
http://localhost/api/documentation
```

Переваги пакету:

- Маршрут захищається middleware (авторизація)
- Підтримує генерацію OpenAPI з PHP-атрибутів (`#[OA\Get(...)]`) — не потрібно підтримувати YAML вручну
- Інтегрується з CI: можна генерувати і валідувати YAML на кожен push

Ключовий меседж: «`l5-swagger` корисний тоді, коли OpenAPI — живий документ, що генерується з коду. Для навчального прикладу мануального HTML достатньо.»

Порівняння підходів

Підхід	Встановлення	Swagger UI URL	OpenAPI-джерело
Мануальний HTML	Нічого не потрібно	`http://localhost/swagger-ui.html`	`public/openapi.yaml`
`darkaonline/l5-swagger`	`composer require` + `vendor:publish`	`http://localhost/api/documentation`	Генерація або готовий YAML

БЛОК 3 → Слайд 9a

Демонстрація: FormRequest — валідація на рівні Laravel

Що показати: файл `app/Http/Requests/Api/V1/Payment/PaymentStoreRequest.php`

```
public function rules(): array
{
    return [
        'account_id'  => ['required', 'integer', 'exists:accounts,id'],
        'amount'      => ['required', 'regex:/^\d+(\.\d{1,2})?$/', 'numeric',
'min:0.01'],
        'currency'    => ['required', 'string', 'in:USD,EUR,UAH'],
```

```
        'description' => ['nullable', 'string', 'max:500'],
    ];
}
```

Ключовий меседж: «FormRequest — це PHP-код, що відповідає за 422 у нашому OpenAPI. Вони мають збігатися. Змінив правило — оновив OpenAPI.»

БЛОК 5 → Слайд 12а

Демонстрація: Feature-тест — happy path 201

Що показати: файл `tests/Feature/Api/V1/CreatePaymentTest.php`

Покажи і прокоментуй структуру:

```
public function test_can_create_payment_with_valid_data(): void
{
    // Arrange — підготовка: акаунт у SQLite in-memory БД
    $account = Account::factory()->create(['balance' => 1000.00]);

    $payload = [
        'account_id'  => $account->id,
        'amount'      => '250.00',
        'currency'    => 'USD',
        'description' => 'Оплата сервісу',
    ];

    // Act — реальний HTTP-запит до /api/v1/payments
    $response = $this->postJson('/api/v1/payments', $payload);

    // Assert — перевіряємо контракт
    $response
        ->assertStatus(201)
        ->assertJsonStructure(['data' => ['id', 'account_id', 'amount', ...]])
        ->assertJsonPath('data.amount', '250.00')
        ->assertJsonPath('data.status', 'processed');

    // Перевіряємо побічний ефект — запис у БД
    $this->assertDatabaseHas('payments', [...]);
}
```

Запустити прямо на миті:

```
docker compose exec php php artisan test tests/Feature/Api/V1/CreatePaymentTest.php
--no-coverage
```

Очікуваний результат:

```
PASS  Tests\Feature\Api\V1\CreatePaymentTest
✓ can create payment with valid data
✓ can create payment without description
```

```
Tests: 2 passed
```

Ключові моменти для пояснення:

- `RefreshDatabase` — кожен тест отримує чисту SQLite `:memory:` БД
- `postJson()` — реальний HTTP-запит через весь стек (Route → FormRequest → Controller → Service → Repository)
- `assertJsonPath('data.amount', '250.00')` — фіксуємо конкретне значення контракту
- `assertDatabaseHas()` — підтверджуємо що дані справді записані в БД

БЛОК 6 → Слайд 14а

Демонстрація: Feature-тести помилок — 422 і 404

Що показати: файл `tests/Feature/Api/V1/CreatePaymentValidationTest.php`

```php
public function test_returns_422_when_required_fields_are_missing(): void
{
    $response = $this->postJson('/api/v1/payments', []);

    $response
        ->assertStatus(422)
        ->assertJsonStructure([
            'message',
            'errors' => ['account_id', 'amount', 'currency'],
        ]);
}
```

Потім покажи: файл `tests/Feature/Api/V1/ShowPaymentNotFoundTest.php`

```php
public function test_returns_404_when_payment_not_found(): void
{
    $response = $this->getJson('/api/v1/payments/999999');

    $response
        ->assertStatus(404)
        ->assertJsonStructure(['message']);
}
```

Запустити:

```
docker compose exec php php artisan test \
  tests/Feature/Api/V1/CreatePaymentValidationTest.php \
  tests/Feature/Api/V1/ShowPaymentNotFoundTest.php \
  --no-coverage
```

Очікуваний результат:

```
PASS  Tests\Feature\Api\V1\CreatePaymentValidationTest
✓ returns 422 when required fields are missing
✓ returns 422 when amount is negative
✓ returns 422 when account does not exist
✓ returns 422 when currency is invalid

PASS  Tests\Feature\Api\V1\ShowPaymentNotFoundTest
✓ returns 404 when payment not found
✓ returns 200 with payment data when found

Tests: 6 passed
```

Де формуються JSON-відповіді помилок — показати `bootstrap/app.php`:

```php
$exceptions->render(function (ValidationException $e, Request $request) {
    if ($request->is('api/v1/*')) {
        return response()->json([
            'message' => 'Validation error Custom',
            'errors'  => $e->errors(),
        ], 422);
    }
});

$exceptions->render(function (NotFoundHttpException $e, Request $request) {
    if ($request->is('api/v1/*')) {
        return response()->json(['message' => 'Record not found Custom'], 404);
    }
});
```

Ключовий меседж: «Якщо хтось змінить ключ `errors` на `validation_errors` — тест впаде. CI заблокує merge. Клієнт захищений.»

БЛОК 7 → Слайд 17a

Демонстрація: Запуск усіх Feature-тестів (CI-сценарій)

Запустити весь Feature suite:

```bash
docker compose exec php php artisan test tests/Feature/Api/V1/ --no-coverage
```

Або весь проєкт:

```bash
docker compose exec php php artisan test --no-coverage
```

Очікуваний результат (нові тести):

```
PASS  Tests\Feature\Api\V1\CreatePaymentTest           2 tests
PASS  Tests\Feature\Api\V1\CreatePaymentValidationTest  4 tests
```

```
PASS  Tests\Feature\Api\V1\ShowPaymentNotFoundTest      2 tests

Tests: 8 passed (49 assertions)
Duration: ~0.25s
```

Покажи таблицю: OpenAPI ↔ Feature-тести

OpenAPI (що має бути)	Feature-тест (що є насправді)
`POST /api/v1/payments` → `201`	`CreatePaymentTest::test_can_create_payment_with_valid_data`
`POST /api/v1/payments` → `422`	`CreatePaymentValidationTest::test_returns_422_*`
`GET /api/v1/payments/{id}` → `404`	`ShowPaymentNotFoundTest::test_returns_404_when_not_found`
`GET /api/v1/payments/{id}` → `200`	`ShowPaymentNotFoundTest::test_returns_200_with_payment_data`

Ключовий меседж: «Кожна відповідь з OpenAPI має відповідний тест. Разом вони — контракт. Тести у CI запускаються на кожен push.»

ПОВНИЙ ПОКАЗ СТЕКА (якщо є час)

Архітектура від HTTP до БД

```
HTTP Request (порт 80, nginx у контейнері php)
    ↓
routes/api.php          Route::apiResource('payments', PaymentController::class)
    ↓
PaymentStoreRequest     Валідація: account_id, amount, currency, description
    ↓
PaymentController       store() → CreatePaymentDTO::fromArray($request-
>validated())
    ↓
PaymentService          createPayment(CreatePaymentDTO $dto) → DB::transaction()
    ↓
PaymentRepository       create(array $data) → Payment::query()->create($data)
    ↓
PaymentResource         toArray() → ['id', 'account_id', 'amount', 'currency', ...]
    ↓
HTTP Response 201 JSON
```

Файли кожного шару:

Шар	Файл
Route	`routes/api.php`
FormRequest	`app/Http/Requests/Api/V1/Payment/PaymentStoreRequest.php`
DTO	`app/DTO/Api/V1/CreatePaymentDTO.php`

Шар	Файл
Controller	`app/Http/Controllers/Api/V1/PaymentController.php`
Service	`app/Services/Api/V1/PaymentService.php`
Repository	`app/Repositories/PaymentRepository.php`
Contract	`app/Contracts/Repositories/PaymentRepositoryInterface.php`
Resource	`app/Http/Resources/PaymentResource.php`
OpenAPI	`public/openapi.yaml`
Feature Tests	`tests/Feature/Api/V1/`

LIVE DEMO: Реальний запит через curl (бонус)

> Сервер вже запущений у контейнері на `http://localhost` (порт 80)

```
# 1. Створити акаунт через tinker
docker compose exec php php artisan tinker --
execute="App\Models\Account::factory()->create(['balance' => 5000])"
# → App\Models\Account { id: 1, balance: 5000, ... }

# 2. Створити платіж — очікуємо 201
curl -s -X POST http://localhost/api/v1/payments \
  -H "Content-Type: application/json" \
  -H "Accept: application/json" \
  -d '{"account_id":1,"amount":"350.00","currency":"USD","description":"Live
demo"}' \
  | python3 -m json.tool

# Очікувана відповідь:
# {
#   "data": {
#     "id": 1,
#     "account_id": 1,
#     "amount": "350.00",
#     "currency": "USD",
#     "description": "Live demo",
#     "status": "processed",
#     "created_at": "2026-04-23 10:00:00"
#   }
# }

# 3. Отримати платіж — очікуємо 200
curl -s http://localhost/api/v1/payments/1 \
  -H "Accept: application/json" | python3 -m json.tool

# 4. Помилка валідації — очікуємо 422
curl -s -X POST http://localhost/api/v1/payments \
  -H "Content-Type: application/json" \
  -H "Accept: application/json" \
  -d '{"amount":"-10","currency":"BTC"}' \
  | python3 -m json.tool

# Очікувана відповідь:
# {
```

```
#   "message": "Validation error Custom",
#   "errors": {
#     "account_id": ["The account id field is required."],
#     "amount": ["The amount must be greater than zero."],
#     "currency": ["The selected currency is invalid."]
#   }
# }

# 5. Неіснуючий платіж — очікуємо 404
curl -s http://localhost/api/v1/payments/999999 \
  -H "Accept: application/json" | python3 -m json.tool
# → { "message": "Record not found Custom" }
```

ЧАСТІ ПИТАННЯ ВІД СТУДЕНТІВ

Q: Чому тести на SQLite, якщо проєкт на MySQL?

> SQLite `:memory:` — ізольована, миттєва, не потребує MySQL-сервера. Для Feature-тестів (HTTP контракт) цього достатньо. Для тестів специфічних для MySQL (FULLTEXT-індекси, JSON-функції) — виносять в окрему тестову MySQL.

Q: Навіщо `RefreshDatabase`?

> Гарантує чисту БД перед кожним тестом. Без нього тести залежать від порядку запуску — "flaky tests".

Q: Чому Feature test, а не Postman?

> Postman = ручний запуск. Feature test = автоматичний, запускається в CI, падіння блокує merge.

Q: Як підтримувати OpenAPI актуальним?

> Правило: «змінив ендпоінт — оновив OpenAPI і тест». Альтернатива: пакет `darkaonline/l5-swagger`, що генерує OpenAPI з PHP-атрибутів прямо в коді. Обидва підходи описані в **БЛОК 2b**.

Q: Як швидко переглянути OpenAPI у браузері?

> Мануально: створи `public/swagger-ui.html` з CDN-посиланням на Swagger UI і відкрий `http://localhost/swagger-ui.html`. Через пакет: `composer require darkaonline/l5-swagger` і відкрий `http://localhost/api/documentation`. Детально — **БЛОК 2b**.

КОМАНДИ-ШПАРГАЛКА

```
# Запустити контейнери
docker compose up -d

# Запустити тільки нові Lesson 7.3 тести
docker compose exec php php artisan test tests/Feature/Api/V1/ --no-coverage

# Запустити всі тести
docker compose exec php php artisan test --no-coverage

# Запустити один файл тесту
docker compose exec php php artisan test tests/Feature/Api/V1/CreatePaymentTest.php
--no-coverage
```

```
# Переглянути маршрути API
docker compose exec php php artisan route:list --path=api/v1

# Застосувати міграції
docker compose exec php php artisan migrate

# Tinker — ручне створення акаунту для live demo
docker compose exec php php artisan tinker --
execute="App\Models\Account::factory()->create(['balance' => 5000])"

# Зайти в контейнер (якщо треба)
docker compose exec php bash

# --- Swagger UI (мануальний варіант) ---
# Відкрити після створення public/swagger-ui.html:
# http://localhost/swagger-ui.html

# --- Swagger UI (через пакет l5-swagger, вже встановлений) ---
# Скинути кеш конфігу після змін у config/l5-swagger.php
docker compose exec php php artisan config:clear
# Відкрити у браузері: http://localhost/api/documentation
```